

Real-Time Game Chat – Algorithms & APIs

1. Ordered Delivery

Core: Per-room Kafka/Pulsar partition for total order; monotonic sequence assigned; clients track lastSeq.

APIs:

- POST /v1/rooms/{roomId}/messages with X-Idempotency-Key
- GET /v1/rooms/{roomId}/messages?afterSeq=...
- WS SUB room:{roomId} {lastSeq}, server sends MSG and GAP frames

2. Transactional Outbox

Core: Write (message, outbox) in one DB txn; relay publishes to broker; idempotent consumers.

APIs:

- Internal outbox writes (DB)
- GET /internal/outbox/lag for ops
- Broker contract: key=roomId, value={messageId, sequenceHint}

3. Hybrid Fanout

Core: Fanout-on-write for small rooms; tree fanout for large rooms.

APIs:

- WS SUB / UNSUB room:{roomId}
- POST /v1/rooms/{roomId}/fanout {mode, targetShards}
- SUBOK frames include shard info

4. TTL Heartbeats

Core: Ping every 15s; TTL 30s; presence stored with sessionId and lastBeat.

APIs:

- WS PING/PONG for heartbeats
- POST /v1/presence/heartbeat {sessionId, status}
- GET /v1/rooms/{roomId}/presence?limit=100

5. Inline + ML Moderation

Core: Inline fast filters; async ML pipeline; message states accepted→under_review→moderated.

APIs:

- POST /v1/rooms/{roomId}/messages → response may return state=under_review
- PATCH /v1/messages/{messageId} {action, reason}
- Webhooks POST {messageId, signals} to moderation tools

6. Graceful Degradation

Core: Drop non-essential events first; rate-limit; read-only fallback.

APIs:

- POST /v1/runtime/mode {scope, id, mode, ttlSec}
- HTTP headers: X-RateLimit-Remaining, Retry-After
- WS control frame: MODE {scope} {mode} {ttlSec}